(Spotify) Blends

Riya Kulkarni
The Information School
University of Washington
riyak30@uw.edu

Andy Nguyen
The Information School
University of Washington
andyn6@uw.edu

Priyanka Kulkarni
The Information School
University of Washington
pkulka2@uw.edu

Dishita Soni
The Information School
University of Washington
dishita@uw.edu

Abstract

This paper presents a group playlist recommendation system built on Spotify extended listening histories for four users. Play events are filtered for genuine engagement using listen duration and skip behavior, and each user is profiled across three representations: a long-term audio feature vector, a TF-IDF weighted genre tag distribution, and mood clusters derived from KMeans. Candidates are assembled from three sources (content k-NN, tag expansion, and neighbor expansion) and re-scored using a hybrid of content and collaborative features under a fixed 80/20 weighting. Re-ranking is performed in two passes: MAP optimization adjusts the personal/group weight split per user, and explicit thumbs up/down feedback drives a final audio-centroid-based re-ranking.

1 Problem Overview

1.1 Problem Statement

Spotify Blend is a group playlist feature that merges the listening histories of multiple users into a single shared playlist. While the concept is appealing, the current implementation falls short in several key ways: it does not accurately reflect users' genuine listening habits, is susceptible to fuzzy intent signals such as marketing placements and social recommendations, and leaves users feeling misrepresented by the algorithm. Music taste is deeply subjective, and a playlist that claims to represent a "blend" of users should meet a higher bar than a simple merge of top tracks.

This project proposes a stricter definition of what a good group playlist should be, grounded in four properties. **Relevance**: every track attributed to a user should genuinely reflect their taste, not an accidental listen: if a user's profile image appears next to a song, they should agree it represents them. **Coherence**: the playlist should have flow and feel intentional, not random. **Equality**: all users in the group should be represented fairly. **Democratic representation**: tracks that multiple users genuinely like should rise to the top.

1.2 Motivation

The core issue with Spotify Blend is that surface-level listening signals are noisy. A song played once at a party, a track recommended by a friend, or an algorithmically auto-played song all register as listens but say very little about a user's actual taste. Fuzzy intentions, like marketing, social recommendations, passive background listening, dilute the concentration of music that genuinely represents a blend of the users, leaving them feeling frustrated and misrepresented by the algorithm.

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

This project addresses that by filtering for genuine engagement, modeling individual taste profiles at multiple levels of granularity, and using both algorithmic optimization and direct user feedback to produce playlists that users would actually stand behind.

1.3 Goals

- Generate personalized candidate pools for each user from a shared listening corpus of 7,068 unique tracks
- Score and rank candidates using a hybrid of content similarity and group engagement signals
- Re-rank results using MAP optimization and refine further through explicit user feedback
- Evaluate playlist quality both quantitatively (MAP, diversity scores) and qualitatively (user ratings)

2 Dataset and Preprocessing

2.1 Dataset

The dataset consists of Spotify extended streaming history for four users (Andy, Dishita, Riya, Priyanka), totaling 29,887 play events across 7,068 unique tracks. [1] For each play, we observe metadata (track name, album artist name) and behavior (timestamp, platform, time played).

We enrich each track with audio features (danceability, energy, valence, tempo, loudness, acousticness, instrumentalness, speechiness, liveness, key, mode, time signature) and genre/tag information sourced from the Last.fm and SoundCharts APIs. Last.fm provides user-generated tags that loosely capture community perceptions of style and mood, while SoundCharts contributes more standardized genre labels and numerical audio descriptors extracted from the underlying audio signal. The resulting dataset combines behavioral logs and content-level descriptors, enabling analysis of both what users play and which musical characteristics best represent their tastes.

2.2 Preprocessing

Raw play events are filtered to retain only track streams via a non-null `spotify_track_uri` and null `episode_name` check. Audio feature columns are flattened from nested JSON and timestamps parsed to UTC-aware datetimes. A play is marked *meaningful* if it was not skipped and `ms_played` $\geq 30{,}000$ms, filtering out accidental plays and brief previews. Per (user, track) pair, we aggregate play count, average `ms_played`, skip rate, and meaningful play fraction.

A scalar play weight is assigned to each (user, track) record:

$$w = \texttt{meaningful_pct} \times \log(1 + \texttt{n_plays}) \tag{1}$$

The log term dampens the effect of repeated plays. Weights are zeroed for tracks where `skip_rate` > 0.70 and `avg_ms` $< 15{,}000$ms, treating consistent early exits as an active dislike signal. These weights are stored as a sparse user $\times$ track matrix used throughout profiling and scoring.

Continuous audio features are normalized to $[0, 1]$ using a fitted `MinMaxScaler`. Genre tags are resolved per track in priority order: `all_tags` $\rightarrow$ `sc_genres` $\rightarrow$ `lastfm_tags`, then encoded as a TF-IDF matrix with `min_df=2` and vocabulary capped at 500 terms.

2.3 User Profiles

Each user is represented by three complementary structures built from the play weight matrix.

Long-term audio profile. A weighted average of normalized audio feature vectors across all listened tracks, producing a 12-dimensional summary of the user's overall sonic taste that emphasizes genuine engagement and suppresses consistent skips.

Tag distribution. An analogous weighted average over the TF-IDF tag matrix, yielding a 500-dimensional profile capturing the genre and mood descriptors most characteristic of each user.

[1] Counts based on the final enriched JSON logs exported from Spotify's "Extended streaming history" user data request.

Pairwise cosine similarities between all four users' tag profiles form a 4×4 similarity matrix used to weight neighbor contributions throughout the pipeline.

Mood clusters. Each user's listening history is clustered into up to four mood states using weighted KMeans over valence, energy, danceability, and tempo. At recommendation time, the active session mood is identified by matching the user's 10 most recently played tracks to their cluster assignments and selecting the plurality cluster.

3 Approach

3.1 Stage 1: Candidate Generation

For each user, up to 1,000 candidate tracks are assembled from three sources. **Content k-NN** queries compute cosine similarity between the user's long-term audio profile and all 7,068 tracks, returning the top 300 matches. Additional queries are issued per mood centroid by splicing centroid values into the long-term profile vector, expanding coverage across the user's different listening moods. Tracks the user consistently skips early (skip rate > 0.70) are excluded. **Tag expansion** ranks all tracks by their TF-IDF scores across the user's top 10 genre tags and selects the top 200 that have been played by at least one group member, grounding genre-based discovery in validated group listening. **Neighbor expansion** identifies tracks that similar users listened to for at least 30 seconds with a skip rate below 0.30, weighting each neighbor's signal by their tag-based cosine similarity to the target user. The top 150 neighbor-sourced tracks are retained, excluding tracks the user has already heard.

The three candidate sets are combined and each track is assigned a pre-score:

$$\text{pre_score} = 0.50 \cdot s_{\text{content}} + 0.20 \cdot s_{\text{tag}} + 0.15 \cdot s_{\text{group}} + 0.15 \cdot s_{\text{neighbor}} \tag{2}$$

The top 1,000 tracks by pre-score are retained per user as the final candidate pool.

3.2 Stage 2: Hybrid Scoring

Each candidate is re-scored using a richer feature set than the pre-score. **Content features** include: long-term audio profile similarity (cosine), tag profile similarity (cosine), active session mood centroid similarity, cluster tag similarity, and tag Jaccard overlap between the track's top 5 tags and the user's top 3 tags. The session mood is determined by finding the mood cluster that best matches the user's 10 most recently played tracks. **Collaborative features** include: group listener count, group average listen duration, group skip rate (inverted), and neighbor-weighted average listen duration. All features are normalized to $[0, 1]$ before combining.

The content score is computed as:

$$s_c = 0.30 \cdot s_{\text{lt}} + 0.25 \cdot s_{\text{tag}} + 0.20 \cdot s_{\text{mood}} + 0.15 \cdot s_{\text{cluster_tag}} + 0.10 \cdot s_{\text{jaccard}} \tag{3}$$

The collaborative score is computed as:

$$s_{cf} = 0.30 \cdot s_{\text{pop}} + 0.25 \cdot s_{\text{ms}} + 0.20 \cdot (1 - s_{\text{skip}}) + 0.25 \cdot s_{\text{nbr_ms}} \tag{4}$$

The final hybrid score is $0.80 \times s_c + 0.20 \times s_{cf}$, producing a ranked pool of 150 tracks per user. The 80/20 split reflects a deliberate design choice to prioritize individual taste while allowing light group signal to influence results. The top 5 hybrid-scored tracks per user are reported in **Table 1.**

3.3 Stage 3: Re-ranking and Playlist Construction

Re-ranking is performed in two passes. First, MAP optimization sweeps over content/collab weight combinations and selects the pair that maximizes Average Precision against each user's ground-truth liked tracks. Second, explicit user feedback is collected via an in-notebook rating interface where each user rates their top 20 songs with thumbs up or thumbs down. Liked tracks pull the final ranking toward similar-sounding songs; disliked tracks push it away. Final playlists are trimmed to 100 tracks per user.

Table 1: Top 5 hybrid-scored tracks per user (w_content=0.80, w_collab=0.20, pool=150)

User	Track	Artist	Hybrid	Pre	Source
Andy	Running Through The Rain	YUGYEOM	0.784	0.708	content, tag
	SUMMERIDE	Jay Park	0.771	0.699	content
	Didn't Cha Know	Erykah Badu	0.767	0.871	tag
	Frontin' (feat. JAY-Z)	Pharrell Williams	0.759	0.896	tag
	Can't Wait	Doja Cat	0.753	0.730	content
Dishita	act ii: date @ 8	4batz	0.741	0.740	content
	Jammin	RUBII	0.733	0.750	tag
	act iv: fckin u (18+)	4batz	0.732	0.736	tag
	Running Through The Rain	YUGYEOM	0.723	0.713	tag
	Out The Way	Dear Silas	0.720	0.663	content
Riya	Charcoal Baby	Blood Orange	0.719	0.693	tag
	My Heart Belongs To U	Jodeci	0.692	0.714	tag
	I'm Not In Love	10cc	0.691	0.680	content, tag
	Ritual Union	Little Dragon	0.680	0.671	tag
	Beat Take 1 (ft. Ghostface)	The Neighbourhood	0.675	0.649	content
Priyanka	Running Through The Rain	YUGYEOM	0.755	0.719	tag
	SUMMERIDE	Jay Park	0.755	0.723	tag
	Party Band (Ft. PUNCHNELLO)	Loco	0.744	0.719	content
	Sundress	A$AP Rocky	0.732	0.739	content
	Late Night	Loco	0.731	0.720	content

4 Evaluation

4.1 Quantitative Evaluation: Mean Average Precision (MAP)

Mean Average Precision is used to evaluate and optimize ranking quality. Ground truth positives are tracks the user listened to for more than 45 seconds on average with a skip rate below 20%. AP scores before and after re-ranking are reported in Table 2.

Table 2: MAP re-ranking results per user

User	Tier	Weights (content/collab)	AP Before	AP After
Andy	fallback-tier1	0.75 / 0.25	0.0057	0.0057
Dishita	primary	0.60 / 0.40	0.0075	0.0152
Riya	primary	0.50 / 0.50	0.0264	0.0352
Priyanka	defaults	0.80 / 0.20	0.0000	0.0000

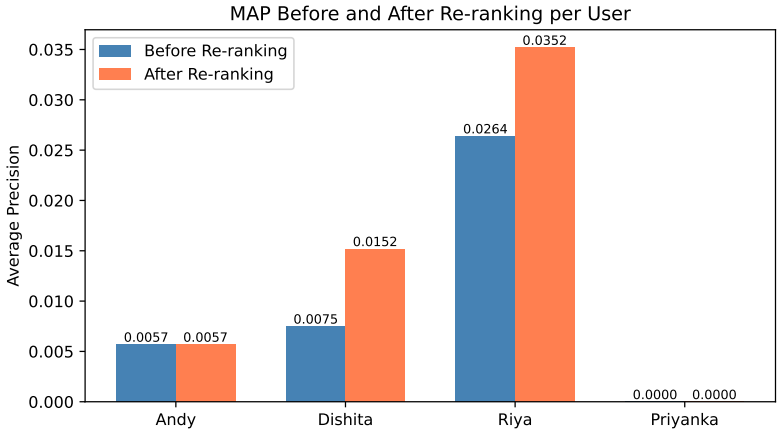

Figure 1: Average Precision before and after re-ranking per user. Dishita and Riya show clear improvement; Andy and Priyanka show no change due to insufficient ground-truth positives.

4.2 Qualitative Evaluation: Explicit User Feedback

Each user rates their top 20 recommended songs using a thumbs up / thumbs down interface built with `ipywidgets` in the Jupyter notebook. Ratings are saved per user and used to produce the final re-ranked playlist. This serves as the primary qualitative signal, directly measuring whether the system output aligns with stated user preferences rather than inferred historical behavior.

5 Results and Discussion

5.1 Results

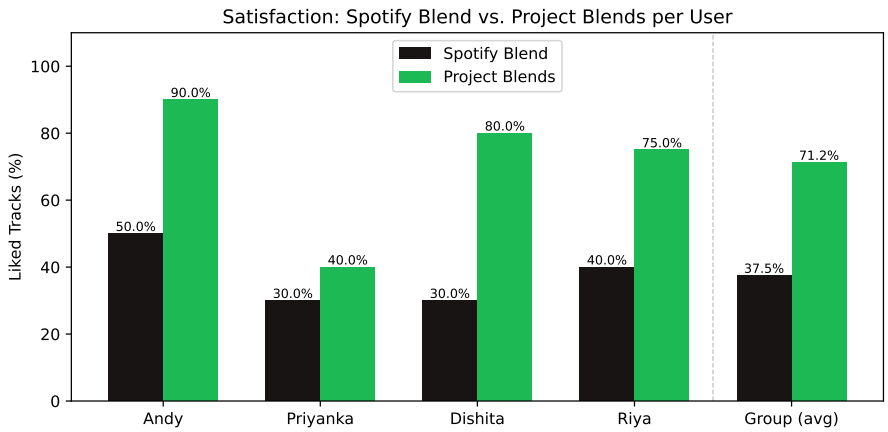

Figure 2: Per-user and group-average like rates on Spotify Blend vs. Project Blends. The rightmost bars (separated by a dashed divider) show the group average. Every user rated the project playlist more favorably, with a group-level gain of 33.8%

Each user rated their top 20 recommended tracks using explicit feedback (thumbs up/thumbs down), and the resulting like rates were compared against ratings collected from the same users on their current Spotify Blend playlist. As shown in Figure 2, the project's system produced higher satisfaction for every user. Dishita showed the largest individual gain (50% increase), followed by Andy (40% increase) and Riya (35% increase). Priyanka showed the smallest improvement (10% increase), consistent with her MAP results in Table 2 where insufficient ground-truth positives limited re-ranking effectiveness. At the group level, the average like rate increased from 37.5% to 71.25%, representing a **33.8% increase in overall satisfaction compared to Spotify Blend.**

These results suggest that the core design decisions of the pipeline meaningfully outperform Spotify's default Blend approach. The low baseline satisfaction with Spotify Blend (37.5% group average) reinforces the motivation stated in Section 1.2: surface-level listening signals produce playlists that users do not feel represent them. The gains for Andy and Dishita in particular indicate that the engagement filtering and content-based profiling were effective at separating genuine taste from passive or accidental listening. Riya's improvement, combined with her MAP gain in Table 2, further validates that the re-ranking stage added meaningful signal on top of the initial hybrid scoring. Priyanka's modest gain highlights the system's primary weakness: when a user's listening history is sparse or noisy enough that no reliable ground-truth signal can be extracted, the re-ranking stage has little to work with and falls back to default weights, limiting personalization.

5.2 Strengths

The pipeline's primary strength is its layered engagement filtering. By zeroing out play weights for consistent early exits and requiring 30 seconds of uninterrupted listening for a play to count as meaningful, the system builds profiles from signals that more reliably reflect genuine taste rather than passive or accidental listening. This distinguishes it from systems that treat all streams equally. Additionally, the three-source candidate generation ensures broad coverage, as any one of them can be adjusted or replaced without breaking the others because each source is independently tunable.

Map re-ranking adds a per-user adaptive layer on top of fixed scoring weights. Instead of using the same content/collab weights for every user, the sweep finds the split that best fits each person's listening history. If a user hasn't listened to enough tracks to produce a reliable signal, the system automatically lowers its threshold and tries again.

5.3 Limitations

The system produces four separate personalized playlists rather than a single shared group playlist, which falls short of the Equal and Democratic goals stated in Section 1.3. Group signal is present throughout the pipeline, but a final aggregation step that resolves individual rankings into one shared list is missing. A true group Blend would require dedicated group aggregation in the re-ranking stage, such as averaging scores across users or applying least-misery scoring. This is the most important direction for future work. With only four users, collaborative filtering signal is weak. The user-user similarity matrix is 4×4, and neighbor expansion draws from at most three listening histories, making group popularity and neighbor features less informative than they would be in a larger system. This likely explains why Andy and Priyanka show no AP improvement in Table 2. The system also has no diversity enforcement, leaving final playlists vulnerable to clustering around a single artist or narrow tag profile. Finally, feedback re-ranking relies on audio centroids of liked and disliked tracks, which cannot capture preference signals unrelated to sound such as lyrics, associations, or overplay.

5.4 Trade-offs

The 80/20 content/collab weight split prioritizes individual taste over group consensus by design. This means a user's playlist will feel personally relevant but may not surface many tracks that the full group shares, which is at odds with the "democratic" goal stated in Section 1.3. A lower content weight would push more group-overlapping tracks to the top but risks playlists that feel less personally representative. The 30-second meaningfulness threshold is conservative. Raising it would produce cleaner engagement signals, but would also discard more of the listening history, thus making profiles sparser especially for users who frequently skip partway through. The current threshold attempts to balance signal quality against data retention, but the right value is inherently subjective and user-dependent.

Code Availability

The full implementation, including data preprocessing, candidate generation, hybrid scoring, MAP re-ranking, and the user feedback interface, is available at `https://github.com/YellowPaco/INFO376-Blends`.

References

1. Belkin, N. J., & Croft, W. B. (1992). Information filtering and information retrieval: Two sides of the same coin? *Communications of the ACM*, *35*(12), 29–38. `https://doi.org/10.1145/138859.138861`
2. Jihoo-Kim. (2020). *RecSys-for-Playlist-Continuation*. GitHub. `https://github.com/jihoo-kim/RecSys-for-Playlist-Continuation`
3. Manning, C. D., Raghavan, P., & Schütze, H. (2009). *Introduction to information retrieval* (Chapter 12: Language models for information retrieval, pp. 237–252). Cambridge University Press. `https://nlp.stanford.edu/IR-book/pdf/12lmodel.pdf`
4. Paraschakis, D., & Nilsson, B. J. (2020). FlowRec: Prototyping session-based recommender systems in streaming mode. In *Advances in Knowledge Discovery and Data Mining: PAKDD 2020*, *12084*, 65–77. `https://doi.org/10.1007/978-3-030-47426-3_6`
5. Ponte, J. M., & Croft, W. B. (1998). A language modeling approach to information retrieval. In *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 275–281. `https://doi.org/10.1145/290941.291008`
6. Sander, D. (2014). *Recommending music on Spotify with deep learning*. sander.ai. `https://sander.ai/2014/08/05/spotify-cnns.html`
7. Schedl, M. (2019). Deep learning in music recommendation systems. *Frontiers in Applied Mathematics and Statistics*, *5*, 44. `https://doi.org/10.3389/fams.2019.00044`

8. Shah, C. (n.d.). *Classical IR: Vector Space Models, Probabilistic Models, Language Models*. Introduction to Search and Recommender Systems. https://drive.google.com/file/d/104jnAduZrXpYCoh_xJCCSjNeUukWkwpA/view

9. Shah, C. (n.d.). *Recommendation Systems*. Introduction to Search and Recommender Systems. https://drive.google.com/file/d/1w7Xi2qub8Ro85x7FA_q0kOf_JyOIJg-r/view

10. Varshneya, K. (2024). *How Spotify recommendation system works and maximises user engagement*. TechAhead. https://www.techaheadcorp.com/blog/spotify-recommendation-system/

11. Wong, J. (2024). Algorithmic symphonies: How Spotify strikes the right chord. *Illumin Magazine*. https://illumin.usc.edu/algorithmic-symphonies-how-spotify-strikes-the-right-chord/